

UNIVERSIDADE FEDERAL DE ITAJUBÁ
ECOX21 - MARATONA DE PROGRAMAÇÃO I
Profs. Edmilson Marmo e Luiz Olmes



Simulado Geral 1

26/08/2026

Regras

1. Há 13 problemas que devem ser resolvidos no tempo estipulado.
2. Os dados de entrada devem ser lidos a partir da entrada padrão.
3. Os dados de saída devem ser escritos na saída padrão.
4. Quando uma linha contém vários valores, eles estarão separados por um único espaço. Não há espaços extras na entrada/saída. Não há linhas em branco na entrada.
5. A entrada e a saída usam o alfabeto latino. Não haverá letras com til, acentos, tremas ou outros sinais diacríticos.
6. Todas as linhas da entrada e da saída, incluindo a última contêm o caractere de fim de linha.
7. O código fonte de cada solução deve ser enviado pelo Boca: `boca.unifei.edu.br`

Ambiente de Testes

A correção das soluções enviadas é realizada no sistema operacional Red Hat Enterprise Linux, versão 8.6 (Ootpa), usando os seguintes compiladores/interpretadores:

C: gcc versão 8.5.0 20210514 (Red Hat 8.5.0-10)
C++: g++ versão 8.5.0 20210514 (Red Hat 8.5.0-10)
Java: openjdk versão 1.8.0_342
Python: python3 versão 3.6.8

Limites

Memória (C, C++, Python): 1GB
Memória (Java): 1GB + 100MB stack
Tamanho máximo do código fonte: 100KB
Tamanho máximo do arquivo executável: 1MB

Códigos que extrapolem os limites permitidos receberão *Runtime Error* como resposta.

Comandos de Compilação

C: gcc -g -O2 -std=gnu11 -static -lm
C++: g++ -g -O2 -std=gnu++17 -static -lm
Java: javac

Códigos C/C++

- O programa deve retornar zero, executando, como último comando, `return 0` ou `exit(0)`.
- Para entradas grandes, objetos `iostream` podem ser lentos, devido a questões de sincronização de buffer com a biblioteca `stdio`. Recomenda-se desabilitar este mecanismo de sincronização em programas que empregam `std::cin` e `std::cout` através dos seguintes comandos:

```
std::ios::sync_with_stdio(0);  
std::cin.tie(0);
```

Note que, neste caso, deve-se evitar usar `printf` e `scanf` no mesmo programa, pois a separação dos buffers pode levar a resultados inesperados.

Códigos Java

- O programa não deve estar encapsulado em um package.
- Para cada problema, o arquivo `.java` e a public class devem ter o mesmo nome `basename` mostrado no Boca.
- Comando de execução: `java -Xms1024m -Xmx1024m -Xss100m`

Códigos Python

- Apenas Python 3 é suportado. Python 3 não é compatível com Python 2.
- **Atenção:** não é garantido que soluções escritas em Python executarão dentro do tempo limite especificado para cada problema.
- Comando de execução: `python3`

Problema $\mathcal{A}$
VIRA CARTAS
Timelimit: 1.0s

José possui um baralho com N cartas. Cada carta possui dois números, um em cada face. Estas cartas são colocadas em uma mesa, com um dos lados virados para cima. Dados dois valores inteiros i e j , com $i \leq j$, uma operação $\text{troca}(i, j)$ consiste em virar todas as cartas da posição i até a posição j , inclusive. Por exemplo, seja a sequência:

valor da face virada para cima: 31 02 45 03 08 01 32 10 04 27 12 07 07 09 63 47
 valor da face virada para baixo: 01 12 06 04 97 02 87 10 03 09 55 56 11 90 03 08

A operação $\text{troca}(5, 11)$ resultaria na sequência abaixo:

valor da face virada para cima: 31 02 45 03 97 02 87 10 03 09 55 07 07 09 63 47
 valor da face virada para baixo: 01 12 06 04 08 01 32 10 04 27 12 56 11 90 03 08

O problema de José é que a sequência de cartas pode ser muito grande e podem ser feitas muitas operações de troca. Ele precisa saber a sequência dos números que estarão virados para cima ao final de todas as operações. Você pode ajudá-lo?

Entrada

A entrada contém vários casos de teste. A primeira linha de um caso de teste contém dois inteiros N e T ($1 \leq N, T \leq 10^5$) indicando, respectivamente, a quantidade de cartas e a quantidade de operações de troca. A segunda linha contém N inteiros v ($0 \leq v \leq 10^9$), indicando os números virados para cima inicialmente. A terceira linha contém N inteiros v , indicando os números virados para baixo inicialmente. As T linhas seguintes contém, cada uma, dois inteiros I e J , indicando os limites de uma operação de troca ($1 \leq I \leq J \leq N$). Valores $N = T = 0$ indicam o fim da entrada.

Saída

Para cada caso de teste, imprima uma linha contendo N inteiros separados por um único espaço, representando os números que estarão virados para cima após todas as operações.

Exemplos

Entrada	Saída
16 1 31 2 45 3 8 1 32 10 4 27 12 7 7 9 63 47 1 12 6 4 97 2 87 10 3 9 55 56 11 90 3 8 5 11 10 5 7 88 23 44 1 67 73 2 9 11 4 55 1 1 3 74 82 9 8 37 1 3 5 10 2 6 5 9 1 7 0 0	31 2 45 3 97 2 87 10 3 9 55 7 7 9 63 47 7 55 1 44 1 67 82 2 9 37

Problema B
ZERO PARA CANCELAR

Timelimit: 1.0s

Seu chefe está ao telefone, nervoso. Ele quer que você compute a soma de uma sequência de números que ele vai falar para você ao telefone, para saber o total das vendas em sua mais recente viagem de negócios. Infelizmente, de vez em quando seu chefe fala números errados para você ao telefone. Felizmente, seu chefe rapidamente percebe que falou um número errado e diz “zero”, que como combinado previamente quer dizer *ignore o último número corrente*. Infelizmente, seu chefe pode cometer erros repetidos, e diz “zero” para cada erro. Por exemplo, seu chefe pode falar ao telefone “Um, três, cinco, quatro, zero, zero, sete, zero, zero, seis”, o que significa uma soma total igual a 7. Para não deixar seu chefe ainda mais nervoso, escreva um programa que determine a soma total dos números falados por seu chefe ao telefone.

Entrada

A primeira linha da entrada contém um inteiro N ($1 \leq N \leq 10^5$), a quantidade de números inteiros (incluindo os “zeros”) que o seu chefe falou ao telefone. Cada uma das N linhas seguintes contém um número inteiro X_i ($0 \leq X_i \leq 100$).

Saída

Seu programa deve produzir uma única linha na saída, contendo um único inteiro, a soma correta dos números, levando em conta que o valor 0 significa erro, conforme descrito.

Exemplos

Entrada	Saída
4 3 0 4 0	0

Entrada	Saída
10 1 3 5 4 0 0 7 0 0 6	7

Problema C
SENHA DA VÓ ZINHA
Timelimit: 1.0s

Vó Zinha foi sempre muito cuidadosa com as senhas que usa para suas atividades na internet, como compras, redes sociais e correio eletrônico, e é especialmente cuidadosa com a senha do banco. No entanto, como está ficando um pouco esquecida das coisas, ela resolveu deixar sua senha do banco escrita, para o caso de necessidade. Obviamente, ela não escreveu simplesmente a senha num papel! Ela inventou uma forma de proteger a senha, mesmo estando escrita, e contou somente para você como fazer para recuperar a senha.

Com um pedaço de papel que Vó Zinha guardou na gaveta onde guarda também suas meias ela fez o seguinte:

- inicialmente escreveu a senha do banco no papel;
- então borrou algumas das letras da senha que tinha escrito de forma que não possam ser lidas;
- para cada uma das letras borradas, ela escreveu no papel uma palavra com K letras;
- por fim, ela escreveu no papel um número inteiro P .

Vó Zinha então contou para você como recuperar a senha:

- utilizando as listas de palavras no papel, substitua cada letra borrada da senha por uma das letras da respectiva lista, obtendo assim possíveis senhas;
- crie uma lista contendo todas as possíveis senhas obtidas no passo anterior;
- ordene a lista de possíveis senhas em ordem lexicograficamente crescente;
- a senha correta é a P -ésima possível senha na lista ordenada.

Por exemplo, considere que no papel esteja escrito (# representa uma letra borrada):

```
x#yy#z  
ab  
cd  
3
```

Fazendo as substituições, a lista das possíveis senhas é $xayycz$, $xbyycz$, $xayydz$, $xbyydz$. Ordenando as possíveis senha obtemos $xayycz$, $xayydz$, **$xbyycz$** , $xbyydz$, e portanto a senha correta é $xbyycz$ (a terceira da lista ordenada).

Hoje Vó Zinha precisa pagar uma conta pela internet e não se recorda da senha do banco. Ela pediu que você pegue o pedaço de papel guardado na gaveta e a ajude a recuperar a senha.

Entrada

A primeira linha da entrada contém três números inteiros N , M e K ($1 \leq N \leq 500$; $1 \leq M \leq N$; $1 \leq K \leq 26$), respectivamente o número de caracteres da senha, o número de letras borradas da senha e o comprimento de cada palavra. A segunda linha contém uma cadeia de caracteres de comprimento N , a senha escrita no papel, com o caractere # (cerquilha) representando as letras borradas. Cada

uma das M linhas seguintes contém uma palavra S_i , sendo que a i -ésima palavra contém as letras para substituir a i -ésima letra borrada da senha. A última linha contém um número inteiro P ($1 \leq P \leq 10^9$), o número de ordem da senha correta na lista ordenada de possíveis senhas.

Saída

Seu programa deve produzir uma única linha, contendo uma única cadeia de caracteres, a senha correta.

Exemplos

<i>Entrada</i>	<i>Saída</i>
6 2 2 x#yy#z ab cd 3	xbyycz

<i>Entrada</i>	<i>Saída</i>
4 1 3 #gof abc 2	bgof

Problema $\mathcal{D}$
TRIO DE BONECAS
Timelimit: 1.0s

Alice é a gerente de produção de uma fábrica de bonecas chinesas. Nessa fábrica, todas as bonecas possuem o mesmo formato, variando apenas de tamanho. As bonecas são vendidas sempre em trios.

Para um trio formado por bonecas de tamanhos A , B e C , com $A \leq B \leq C$, o *balanceamento* do trio é definido como $(A - B)^2$. De acordo com o controle de qualidade da fábrica, idealmente cada trio deveria ter balanceamento 0, ou seja, conter duas bonecas de mesmo tamanho e uma terceira boneca de tamanho maior ou igual a essas duas; porém, isso nem sempre é possível devido a limitações de estoque.

Alice acabou de receber um pedido urgente: o consulado Chinês fará um grande evento no qual pretende distribuir K trios de bonecas, e precisa que Alice envie imediatamente todos os trios. Existem na fábrica N bonecas de diversos tamanhos, ainda não separadas em trios, e Alice não sabe se conseguirá formar K trios ideais.

Devido à urgência do evento, Alice e o consulado fizeram um acordo: o consulado aceitará trios que não sejam ideais; porém, para garantir um bom evento, Alice deve enviar uma caixa com K trios de bonecas cuja soma dos balanceamentos seja a menor possível.

Por exemplo, suponha que Alice possua 7 bonecas com tamanhos 11, 1, 7, 5, 16, 8 e 15, e que o consulado tenha solicitado 2 trios de bonecas. Algumas configurações possíveis para envio são:

- **Configuração 1:** trios (7, 8, 15) e (1, 5, 16).
 - O primeiro trio possui balanceamento $(8 - 7)^2 = 1$
 - O segundo trio possui balanceamento $(5 - 1)^2 = 16$
 - O balanceamento total dessa configuração é $1 + 16 = 17$
- **Configuração 2:** trios (5, 7, 16) e (8, 11, 15).
 - O primeiro trio possui balanceamento $(7 - 5)^2 = 4$
 - O segundo trio possui balanceamento $(11 - 8)^2 = 9$
 - O balanceamento total dessa configuração é $4 + 9 = 13$

Existem outras configurações possíveis, mas a menor soma de balanceamentos possível é 13 (correspondente à Configuração 2).

Ajude Alice a calcular a soma de balanceamentos mínima em um conjunto com K trios de bonecas.

Entrada

A primeira linha de entrada contém dois inteiros N e K ($3 \leq N \leq 10000$; $1 \leq K \leq 3000$; $3K \leq N$), representando, respectivamente, o número de bonecas e a quantidade de trios solicitados pelo consulado Chinês. A segunda linha contém N números inteiros. O i -ésimo desses números será T_i ($1 \leq T_i \leq 100000$; $1 \leq i \leq N$), representando o tamanho da i -ésima boneca

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, a menor soma de balanceamentos possível em uma caixa com K trios de bonecas.

Exemplos

Entrada	Saída
7 2 11 1 7 5 16 8 15	13

Entrada	Saída
8 2 2 1 2 2 2 1 2 2	0

Problema E**BARALHO**

Timelimit: 1.0s

Uma gráfica iniciou a produção de cartas de baralho. Cada baralho produzido deve ser um *baralho completo*, ou seja, deve ter exatamente 52 cartas, compreendendo quatro naipes (Copas, Espadas, Ouros e Paus), com treze cartas em cada naipe (Ás, 2, 3, 4, 5, 6, 7, 8, 9, 10, Valete, Dama e Rei). Um robô coleta cartas produzidas pelas máquinas impressoras e cortadoras e as agrupa em conjuntos de 52 cartas, preparando o baralho para ser embalado para venda. A empresa deseja garantir que cada baralho embalado seja um *baralho completo* e precisa de sua ajuda. Dada a lista das cartas de um baralho pronto para ser embalado, escreva um programa para verificar se há cartas faltando ou duplicadas no baralho.

Entrada

A primeira linha da entrada contém uma cadeia de caracteres (entre 3 e 156 caracteres, inclusive) que descreve as cartas do baralho. Cada carta é descrita usando três caracteres, no formato ddN onde dd são dois dígitos decimais (de 01, representando a carta Ás, a 13, representando a carta Rei) e N é um caractere entre C, E, U e P (representando os naipes Copas, Espadas, Ouros e Paus). Note que o caractere que representa o naipe Ouros é U (e não O), para não confundir com o dígito zero.

Saída

Seu programa deve produzir exatamente quatro linhas na saída, cada linha correspondendo aos naipes Copas, Espadas, Ouros, e Paus, nessa ordem. Para cada naipe, se o conjunto de cartas está completo (ou seja, se exatamente 13 cartas com valores de 01, 02, 03, ..., 12, 13 estão presentes), seu programa deve produzir o valor 0; se o conjunto de cartas tem alguma carta duplicada, seu programa deve produzir a palavra erro; se o conjunto de cartas tem cartas faltando, seu programa deve imprimir o número de cartas que faltam.

Exemplos

Entrada	Saída
11P01C02C01U02U03U04U	11 13 9 12

Entrada	Saída
13P02P01P03P04P05P06P07P08P09P10P11P12P	13 13 13 0

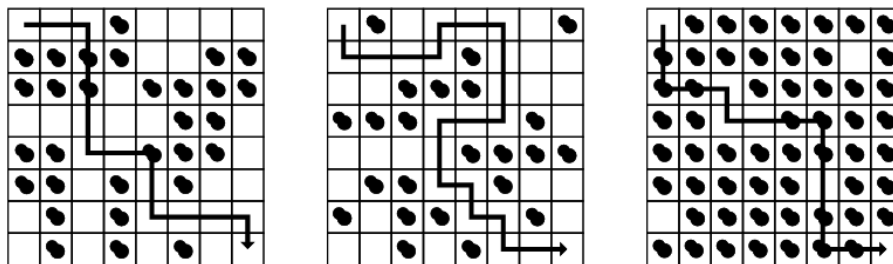
Entrada	Saída
01C02C03C04C05C07C09C10C11C02E02E03E11U	4 erro 12 13

Problema $\mathcal{F}$

MINA

Timelimit: 1.0s

Uma mina de ouro é representada por N linhas e N colunas de quadrados. O mineiro está no quadrado inicial (superior esquerdo) e precisa cavar até o quadrado final (inferior direito), onde existe a maior concentração de ouro da mina. Alguns quadrados, porém, estão bloqueados por pedras, o que dificulta o trabalho. Sabendo que o mineiro pode realizar apenas movimentos ortogonais, seu programa deve calcular o número mínimo de quadrados bloqueados pelos quais o mineiro tem que passar para chegar no quadrado inferior direito. Os quadrados inicial e final nunca estão bloqueados. A figura abaixo ilustra três possíveis minas, para $N = 8$, para as quais os números mínimos de quadrados bloqueados são, respectivamente, três, zero e nove. A figura também mostra três possíveis trajetórias mínimas, como exemplo.



Entrada

A primeira linha da entrada contém um inteiro N ($2 \leq N \leq 100$), representando as dimensões da mina. Cada uma das N linhas seguintes contém N inteiros, definindo os quadrados da mina. O inteiro 0 (zero) representa um quadrado livre e o inteiro 1, um quadrado bloqueado.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o número mínimo de quadrados bloqueados pelos quais o mineiro tem que passar para chegar no quadrado final.

Exemplos

Entrada	Saída
6 0 1 0 0 0 0 1 1 0 0 1 1 1 0 1 1 1 1 0 0 0 1 1 0 0 0 1 1 1 0 0 1 0 0 0 0	3

Entrada	Saída
2 0 0 1 0	0

Problema $\mathcal{G}$

FAST-FOOD

Timelimit: 1.0s

Após a descoberta de um tesouro escondido terminar uma maldição milenar, a histórica ilha da Quadradônia está pronta para ser repovoada. A expectativa mundial é de que a ilha se torne um grande ponto turístico, recebendo visitantes de todo o planeta. Como você pode imaginar, a Quadradônia tem esse nome porque as suas ruas formam um grid quadriculado e todos os prédios comerciais estão localizados em intersecções entre duas ruas.

Você foi contratado como programador na *Fast-Food Turbo* (FFT), uma rede de franquias de restaurantes de comida rápida. De olho nas oportunidades abertas na Quadradônia, a FFT adquiriu N prédios comerciais na ilha, e deseja abrir, nesses prédios, unidades de duas das suas redes mais populares: o restaurante de hambúrgueres quadrados Quaburguer e o restaurante de pizzas quadradas Pizzatro.

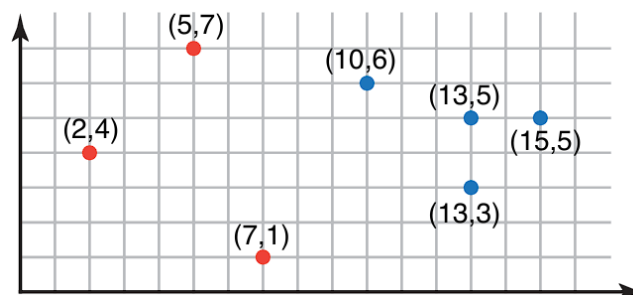
Devido à localização remota da ilha, é muito difícil levar mercadorias para a Quadradônia. Por isso, quando faltam ingredientes em um restaurante da ilha, a solução mais rápida sempre é pedir esses ingredientes a algum outro restaurante da ilha. Se outro restaurante tiver os ingredientes necessários, ele manda um funcionário de carro para fazer a entrega ao restaurante necessitado. Porém, como o Quaburguer e a Pizzatro usam ingredientes bem diferentes, um restaurante só pode fornecer ingredientes para outro restaurante da mesma rede.

Como não existem engarrafamentos na Quadradônia, o tempo que um entregador leva para ir de um prédio a outro é sempre proporcional à *distância quadradônica* entre os prédios, definida como a distância que precisa ser percorrida em ruas da ilha para ir de um prédio a outro. Mais formalmente, se os prédios i e j têm coordenadas (X_i, Y_i) e (X_j, Y_j) , respectivamente, a distância quadradônica entre eles é definida como:

$$d_{quad}(i, j) = |X_i - X_j| + |Y_i - Y_j|$$

A FFT precisa decidir, para cada prédio, de qual rede o restaurante desse prédio fará parte. Como a empresa não consegue prever quais prédios precisarão de ingredientes, os membros do conselho decidiram que eles querem minimizar o pior caso possível, ou seja, a distância d_{quad} máxima entre todos os pares de restaurantes de uma mesma rede.

A figura abaixo ilustra um exemplo para $N = 7$. Observe que, se os prédios 1 e 4 pertencerem à mesma rede, a resposta será no mínimo a distância entre eles, $d_{quad}(1, 4) = |2 - 13| + |4 - 5| = 12$. Porém, podemos conseguir uma distância máxima igual a 8 se tivermos uma rede com os prédios (1, 3, 5) e a outra com os prédios (2, 4, 6, 8):



Existem outras configurações possíveis, mas 8 é a distância mínima para esse conjunto de prédios.

Sua tarefa é: dadas as coordenadas dos N prédios que a FFT adquiriu, distribua os N prédios entre as duas redes de modo a minimizar a distância quadradônica máxima entre dois restaurantes de uma mesma rede.

Entrada

A primeira linha da entrada contém um único inteiro N ($1 \leq N \leq 300000$), o número de prédios da Quadradônia adquiridos pela FFT. A segunda linha da entrada contém N inteiros. O i -ésimo desses inteiros, X_i ($1 \leq X_i, Y_i \leq 500000000$; $1 \leq i \leq N$), representa a primeira coordenada do i -ésimo prédio. A terceira e última linha da entrada contém N inteiros. O i -ésimo desses inteiros, Y_i , representa a segunda coordenada do i -ésimo prédio. É possível que dois prédios tenham as mesmas coordenadas.

Saída

Seu programa deverá produzir uma única linha contendo um único inteiro, a menor distância quadradônica máxima possível entre dois restaurantes de uma mesma rede.

Exemplos

<i>Entrada</i>	<i>Saída</i>
7 2 10 7 13 5 13 15 4 6 1 5 7 3 5	8

Problema $\mathcal{H}$
TORNEIO DE TÊNIS
Timelimit: 1.0s

A prefeitura contratou um novo professor para ensinar as crianças do bairro a jogar tênis na quadra de tênis do parque municipal. O professor convidou todas as crianças do bairro interessadas em aprender a jogar tênis. Ao final do primeiro mês de aulas e treinamentos foi organizado um torneio em que cada participante disputou exatamente seis jogos. O professor vai usar o desempenho no torneio para separar as crianças em três grupos, de forma a ter grupos de treino em que os participantes tenham habilidades mais ou menos iguais, usando o seguinte critério:

- participantes que venceram 5 ou 6 jogos serão colocados no Grupo 1;
- participantes que venceram 3 ou 4 jogos serão colocados no Grupo 2;
- participantes que venceram 1 ou 2 jogos serão colocados no Grupo 3;
- participantes que não venceram nenhum jogo não serão convidados a continuar.

Dada uma lista com o resultado dos jogos de um participante, escreva um programa para determinar em qual grupo ele será colocado.

Entrada

A entrada consiste de seis linhas, cada linha indicando o resultado de um jogo do participante. Cada linha contém um único caractere: V se o participante venceu o jogo, ou P se o jogador perdeu o jogo. Não há empates nos jogos.

Saída

Seu programa deve produzir uma única linha na saída, contendo um único inteiro, identificando o grupo em que o participante será colocado. Se o participante não for colocado em nenhum dos três grupos seu programa deve imprimir o valor -1 .

Exemplos

<i>Entrada</i>	<i>Saída</i>
V V P P P V	2

<i>Entrada</i>	<i>Saída</i>
P P P P P P	-1

Problema I RETÂNGULO

Timelimit: 1.0s

Vô Pedro é um fazendeiro meticuloso. Em sua fazenda ele tem uma plantação no formato circular, com algumas árvores plantadas exatamente na circunferência da plantação. A figura (a) abaixo mostra a plantação com as árvores.

Agora vô Pedro quer usar uma longa corda e quatro das árvores para demarcar um retângulo na plantação, usando as árvores como vértices, com a corda marcando os lados. A figura (b) abaixo mostra dois retângulos que podem ser demarcados usando as árvores na plantação figura (a).



Dada a descrição das posições das árvores na plantação circular de vô Pedro, sua tarefa é determinar se é possível demarcar um retângulo conforme descrito acima.

Entrada

A primeira linha da entrada contém um inteiro N ($4 \leq N \leq 10^5$) indicando o número de árvores na circunferência da plantação. As árvores são representadas como pontos na circunferência. A segunda linha contém N inteiros $L_1, L_2, \dots, L_N$ ($1 \leq L_i \leq 10^6$; $1 \leq i \leq N$), indicando o comprimento do arco entre cada par de árvores consecutivas. Os arcos são dados no sentido anti-horário.

Saída

Seu programa deve produzir uma única linha, contendo um único caractere, que deve ser S se é possível demarcar um retângulo usando as árvores como vértices, ou N caso contrário.

Exemplos

Entrada	Saída
8 3 3 4 2 6 2 2 2	S
Entrada	Saída
4 14 16 15 15	N
Entrada	Saída
6 3 7 7 3 10 10	S

Problema 7
POLIGRAMA
Timelimit: 1.0s

Duas palavras A e B são *anagramas entre si* se podemos transformar a palavra A na palavra B apenas trocando de posição as letras da palavra A . Por exemplo, “duetos” e “estudo” são anagramas entre si. Um outro exemplo é “bba” e “bab”.

Vamos chamar de poligrama uma palavra que consiste na concatenação de duas ou mais palavras que são anagramas entre si. A primeira dessas palavras é chamada de raiz do poligrama. Por exemplo, a palavra “bbabab” é um poligrama com raiz “bba”, pois ela é a concatenação dos anagramas “bba” e “bab”.

Dada uma palavra, escreva um programa que determine se ela é um poligrama e encontre a sua raiz.

Entrada

A primeira linha da entrada contém um inteiro N ($1 \leq N \leq 100000$), indicando o número de letras da palavra. A segunda linha contém a palavra P . Os únicos caracteres em P são letras minúsculas não acentuadas.

Saída

Seu programa deve produzir uma única linha. Se a palavra dada é um poligrama, a linha deve conter a raiz do poligrama. Caso contrário, a linha deve conter o caractere asterisco (*). Se houver mais de uma raiz possível, seu programa deve imprimir a de menor comprimento.

Exemplos

Entrada	Saída
5 xxxxx	x

Entrada	Saída
2 xy	*

Entrada	Saída
6 bbabab	bba

Problema $\mathcal{K}$
CANDIDATAS
Timelimit: 1.0s

Carla é uma biotecnóloga que está investigando proteínas para usar em uma vacina. As proteínas estão codificadas em uma longa sequência de números naturais, que vamos chamar de S .

Através de múltiplos experimentos, Carla determinou que uma subsequência contígua de S é *candidata* se o máximo divisor comum dos elementos da subsequência é maior do que 1.

Carla quer investigar o número de subsequências candidatas contidas em alguns intervalos contíguos de S , possivelmente também alterando alguns elementos de S . Mais precisamente, durante suas pesquisas ela deseja realizar operações de dois tipos:

1. alterar o valor de um elemento da sequência S ; e
2. consultar o número de subsequências candidatas em um dado trecho contíguo de S .

Ajude Carla a realizar suas pesquisas e avançar na busca pela vacina!

Entrada

A primeira linha da entrada contém dois inteiros N e M ($1 \leq N, M \leq 10^5$), indicando respectivamente o número de elementos na sequência S e o número de operações a serem realizadas. A segunda linha contém N inteiros S_i ($1 \leq S_i \leq 10^9$; $1 \leq i \leq N$), os elementos da sequência S . Os elementos são identificados por índices de 1 a N . Cada uma das M linhas seguintes descreve uma operação e contém três inteiros. O primeiro inteiro, T , indica o tipo de operação e pode ser 1 ou 2. Se a operação é do tipo 1, os dois números seguintes na linha são I e V ($1 \leq I \leq N$; $1 \leq V \leq 10^9$), indicando que o elemento de índice I da sequência S deve ter o valor atualizado para V . Se a operação é do tipo 2, os dois números seguintes na linha são E e D ($1 \leq E \leq D \leq N$), indicando respectivamente o índice do elemento inicial e o índice do elemento final de um intervalo contíguo da sequência S .

Saída

Para cada operação do tipo 2, seu programa deve produzir uma única linha, contendo um único inteiro, o número de subsequências candidatas no intervalo indicado.

Exemplos

<i>Entrada</i>	<i>Saída</i>
5 1 9 3 4 8 1 2 2 5	4
<i>Entrada</i>	<i>Saída</i>
4 3 4 4 4 4 2 1 4 1 3 5 2 1 4	10 5

Problema $\mathcal{L}$
TEMPO DE RESPOSTA
Timelimit: 1.0s

Sara adora trocar mensagens com amigos. Como ela recebe e envia muitas mensagens, está preocupada com o tempo que seus amigos esperam para receber respostas das mensagens. As seguintes regras de etiqueta são sempre obedecidas:

- as únicas mensagens que Sara envia são respostas a mensagens que ela recebeu.
- Sara envia no máximo uma mensagem como resposta a uma mensagem que recebeu.
- um amigo de Sara nunca envia uma nova mensagem para Sara até que tenha recebido resposta da mensagem que enviou anteriormente.

O aplicativo de mensagens que Sara e seus amigos usam recebe e envia mensagens instantaneamente. O envio e o recebimento de mensagens são chamados de *eventos*. O aplicativo registra cada evento na ordem em que os eventos ocorrem, usando dois tipos de registro:

- $R X$ indica que uma mensagem foi recebida do amigo X .
- $E X$ indica que uma mensagem foi enviada ao amigo X .

O aplicativo usa ainda um outro tipo de registro, para indicar o tempo que se passou entre dois eventos consecutivos, na forma:

- $T X$ indicando que X segundos se passaram entre o evento anterior e o evento posterior a esse registro.

Se não há registro do tipo $T X$ entre dois registros de eventos consecutivos significa que exatamente 1 segundo se passou entre esses dois eventos.

O *Tempo de Resposta* de uma mensagem é o tempo que se passa entre o recebimento da mensagem por Sara e o envio da resposta a essa mensagem por Sara. Se um amigo recebeu respostas para todas as suas mensagens, o *Tempo de Resposta Total* para esse amigo é a soma dos Tempos de Respostas para as mensagens desse amigo; caso contrário o Tempo de Resposta Total para esse amigo é -1 .

Dada a lista de registros do aplicativo de Sara, sua tarefa é determinar o Tempo de Resposta Total para cada amigo.

Entrada

A primeira linha da entrada contém um inteiro N ($1 \leq N \leq 20$), o número de registros. Os amigos de Sara são identificados por números inteiros. Cada uma das N linhas seguintes descreve um registro e contém um caractere (R , E ou T) seguido de um número inteiro X ($1 \leq X \leq 100$). No caso de registros dos tipos R e E o valor de X indica um amigo de Sara; no caso do registro de tipo T , o valor de X indica o número de segundos que se passaram entre o evento anterior e o posterior.

Saída

Para cada amigo de Sara seu programa deve produzir uma linha na saída contendo dois inteiros: o número do amigo e o Tempo de Resposta Total para esse amigo, em ordem crescente dos números dos amigos.

Exemplos

<i>Entrada</i>	<i>Saída</i>
5	2 6
R 2	3 6
R 3	
T 5	
E 2	
E 3	

<i>Entrada</i>	<i>Saída</i>
14	12 13
R 12	23 8
T 2	34 2
R 23	45 -1
T 3	
R 45	
E 45	
R 45	
E 23	
R 23	
T 2	
E 23	
R 34	
E 12	
E 34	

Problema $\mathcal{M}$

TRANSPORTES

Timelimit: 1.0s

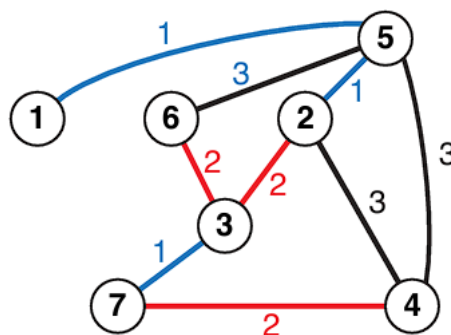
Ênia, a atual rainha da Nlogônia, é fascinada por meios de transporte de todos os tipos. Como consequência, a Nlogônia possui a melhor rede de transporte público do mundo inteiro.

Existem N estações espalhadas por todo o reino, K sistemas de transporte (metrô, ônibus, avião, barco, etc) e M ligações diretas entre pares de estações. Cada ligação pertence a um único sistema de transporte entre os K existentes, e pode ser percorrida nas duas direções. Para cada par de estações, existe no máximo uma ligação no total (ou seja, entre todos os sistemas).

Para usar os sistemas de transporte, o usuário precisa comprar passagens. Ênia estabeleceu as seguintes regras para estimular o transporte público:

- Cada um dos K sistemas de transporte tem um preço fixo de passagem. Assim, o preço de uma passagem do sistema i é P_i , independente da estação.
- A passagem só deve ser paga quando o usuário entrar no sistema de transporte.
- O usuário só pode entrar em um sistema por vez (por exemplo, para entrar no sistema de avião ele precisa sair do sistema de metrô).
- Após pagar o preço P_i para entrar no sistema i , o usuário pode se locomover o quanto desejar usando ligações do sistema i , contanto que ele não saia desse sistema. Caso o usuário troque de sistema, ele deverá pagar por uma nova passagem de preço P_i para retornar ao sistema i .

Por exemplo, considere a seguinte rede de transportes:



Note que há 7 estações e 9 ligações, das quais 3 são do sistema 1, 3 são do sistema 2 e 3 são do sistema 3. Suponha que os preços das passagens sejam $P_1 = 5$, $P_2 = 1$ e $P_3 = 7$ e que o usuário deseja fazer uma viagem da estação 1 para a estação 7, o menor custo total entre todas as possíveis viagens é 11:

- $1 \rightarrow 5$: usa o sistema 1, então pagamos $P_1 = 5$.
- $5 \rightarrow 2$: também usa o sistema 1, então não pagamos nada.
- $2 \rightarrow 3$: usa o sistema 2, então pagamos $P_2 = 1$ para trocar de sistema.
- $3 \rightarrow 7$: usa o sistema 1, então pagamos $P_1 = 5$ para voltar ao sistema 1.
- No total, essa viagem custará: $5 + 1 + 5 = 11$.

Ênia deseja saber o menor custo total para ir da estação A para a estação B , e para isso te contratou. Responda corretamente, ou ela te expulsará da Nlogônia!

Entrada

A primeira linha da entrada contém três inteiros N , M e K ($2 \leq N \leq 10^5$; $1 \leq M, K \leq 10^5$), representando o número de estações, o número de ligações entre as estações, e a quantidade de sistemas de transporte, respectivamente.

A segunda linha contém K números inteiros. O i -ésimo desses números é P_i ($1 \leq P_i \leq 1000$; $1 \leq i \leq K$), representando o preço da passagem do sistema de transporte i . As próximas M linhas contém a descrição das ligações. A j -ésima dessas linhas tem três números V_j , U_j e T_j ($1 \leq V_j, U_j \leq N$; $1 \leq j \leq M$; $V_j \neq U_j$; $1 \leq T_j \leq K$), indicando que há uma ligação entre as estações V_j e U_j usando o sistema T_j .

A última linha da entrada contém dois números inteiros, A e B ($1 \leq A, B \leq N$; $A \neq B$), representando a estação inicial e a estação final da viagem requisitada pela rainha.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o menor custo total de uma viagem que começa na estação A e termina na estação B . Se não for possível fazer uma viagem de A a B usando a rede de transportes, imprima -1 .

Exemplos

Entrada	Saída
6 4 1 50 1 4 1 6 1 1 3 2 1 4 6 1 3 5	-1

Entrada	Saída
5 5 2 1 1 1 2 1 5 2 1 1 3 2 4 3 1 4 5 2 2 3	2